

Solana Attestation Service

Security Assessment

May 8th, 2025 — Prepared by OtterSec

Xiang Yin	soreatu@osec.io
Gabriel Ottoboni	ottoboni@osec.io
Kevin Chow	kchow@osec.io
Robert Chen	r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-SAS-ADV-00 | Re-utilization of Attestation Account 6

OS-SAS-ADV-01 | Failure to Check Pause Status of Schemas 7

OS-SAS-ADV-02 | Inadequate Rent Funding on Increased Account Size 8

General Findings **10**

OS-SAS-SUG-00 | Missing Ownership Checks 11

OS-SAS-SUG-01 | Improper Validation Logic 12

OS-SAS-SUG-02 | Code Maturity 13

OS-SAS-SUG-03 | Unutilized/Redundant Code 15

Appendices

Vulnerability Rating Scale **17**

Procedure **18**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Solana Foundation engaged OtterSec to assess the `solana-attestation-service` program. This assessment was conducted between April 21st and April 30th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we identified a vulnerability where changing the authorized signers or schema description may grow account size without adding enough lamports for rent-exemption, resulting in runtime errors (OS-SAS-ADV-02). Additionally, re-utilizing a closed attestation account with the same nonce fails system account verification (OS-SAS-ADV-00). Furthermore, when creating an attestation, the current logic fails to check if the paused flag is set, allowing attestations to be created even when the schema is paused (OS-SAS-ADV-01).

We made recommendations for implementing proper validations (OS-SAS-SUG-01) and ownership checks, particularly during the creation of a schema and while changing the schema version (OS-SAS-SUG-00). We further suggested adhering to coding best practices (OS-SAS-SUG-02) and advised removing redundant or unutilized code instances (OS-SAS-SUG-03).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/solana-foundation/solana-attestation-service. This audit was performed against commit dda0ab2.

A brief description of the program is as follows:

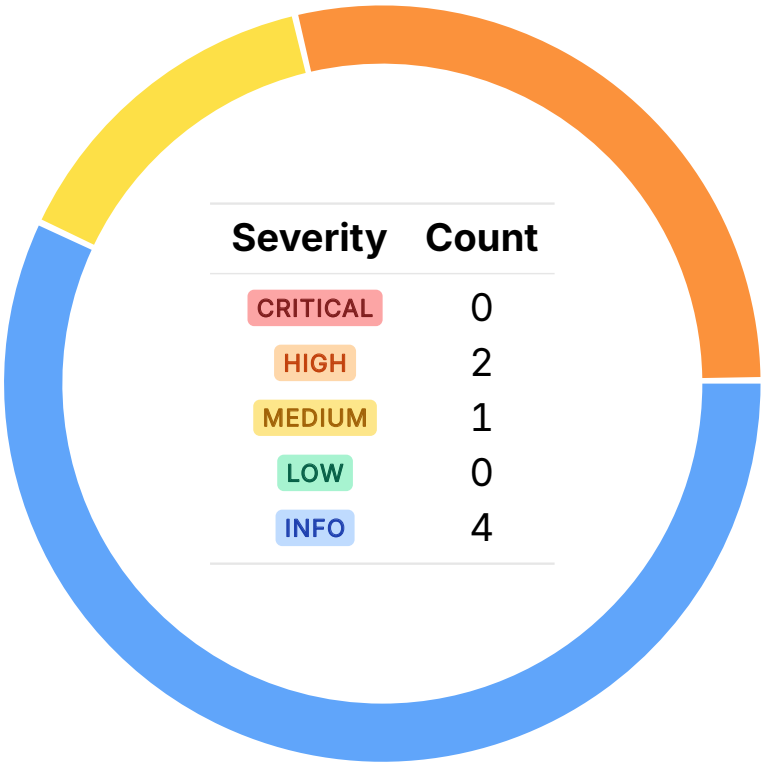
Name	Description
solana-attestation-service	A system for creating, managing, and verifying attestations on the Solana blockchain. It allows entities to define credentials using schemas, issue attestations based on those schemas, and optionally tokenize them using Solana's native token features.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	2
MEDIUM	1
LOW	0
INFO	4

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-SAS-ADV-00	HIGH	RESOLVED	Re-utilizing a closed attestation account with the same nonce fails `verify_system_account`.
OS-SAS-ADV-01	HIGH	RESOLVED	`create_attestation` ignores the `is_paused` flag, allowing attestations to be created even when the `schema` is paused.
OS-SAS-ADV-02	MEDIUM	RESOLVED	The instructions `change_authorized_signers` and `change_schema_description` may grow account size without adding enough lamports for rent-exemption, resulting in runtime errors. Tests missed this because they did not trigger actual account growth.

© 2025 Otter Audits LLC. All Rights Reserved.

Re-utilization of Attestation Account HIGH OS-SAS-ADV-00

Description

If an attestation is closed and then re-utilized with the same nonce and account, the `verify_system_account` check in `process_create_attestation` will fail because the expected token account cannot be initialized as it already is.

Remediation

Ensure that the code does not rely on a token account not existing.

Patch

Resolved in 96a7936.

© 2025 Otter Audits LLC. All Rights Reserved.

Failure to Check Pause Status of Schemas HIGH OS-SAS-ADV-01

Description

`create_attestation` fails to check whether the associated `Schema` is paused, rendering the `is_paused` flag ineffective. This allows attestations to be created even when the `schema` is explicitly disabled, breaking expected governance and control mechanisms.

Remediation

Add a conditional check that aborts attestation creation if `schema.is_paused` is true.

Patch

Resolved in 8e59107.

© 2025 Otter Audits LLC. All Rights Reserved.

Inadequate Rent Funding on Increased Account Size MEDIUM OS-SAS-ADV-02

Description

There is a latent vulnerability in how account reallocation and rent-exemption are handled in `change_authorized_signers` and `change_schema_description` (highlighted below), which modify account data by reallocating the account to a larger size without adjusting its lamport balance. In Solana, accounts must always maintain rent-exemption based on their current size, and increasing the size increases the rent requirement. If the lamports are not topped up accordingly during a `realloc`, the account becomes non-rent-exempt, resulting in an `InsufficientFundsForRent` error during execution.

>_ *src/processor/change_authorized_signers.rs* RUST

```rust
#[inline(always)]
pub fn process_change_authorized_signers([...]) -> ProgramResult {
    [...]
    if new_space != credential_info.data_len() {
        credential_info.realloc(new_space, false)?;
    }
    [...]
}
```

The tests failed to catch this because they either shrink the account before growing it again (so the net size does not increase), as done in `change_authorized_signers_success`, or fail to grow the account, as done in `change_schema_description_success`, implying no actual increase in rent requirement occurs. As a result, the missing lamport top-up is never exercised in tests. In production, however, users may trigger actual account growth by adding more signers or updating a schema with a longer description, resulting in the program failing.

>_ *src/processor/change_schema_description.rs* RUST

```rust
#[inline(always)]
pub fn process_change_schema_description([...]) -> ProgramResult {
    [...]
    // Resize account if needed.
    let new_description_len = schema.description.len();
    if new_description_len != prev_description_len {
        let new_space = schema_info.data_len() + new_description_len - prev_description_len;
        schema_info.realloc(new_space, false)?;
    }
    [...]
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Ensure to calculate the new rent-exempt threshold via `Rent::minimum_balance(new_size)` when reallocating an account to a larger size.

Patch

Resolved in 1b882d5.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-SAS-SUG-00	There are several instances where there is a lack of proper ownership checks, particularly during the creation of a schema and while changing the schema version.
OS-SAS-SUG-01	There are several instances where proper validation is not performed, resulting in potential issues.
OS-SAS-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-SAS-SUG-03	The codebase contains multiple cases of redundant or unutilized code that should be removed.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Ownership Checks OS-SAS-SUG-00

Description

1. `create_schema` should validate that the `credential_info` account is owned by the current program (`program_id`). Without this check, an externally owned `Credential` account may be utilized as `credential_info` in `process_create_schema`, allowing for a schema with an invalid `credential` field to be created. There is no immediate impact, as it is not possible to utilize the schema in another instruction.
2. `change_schema_version` does not validate that the `existing_schema` is actually linked to the provided `credential_info` account. This omission allows a user to potentially version-bump a schema that does not belong to their `credential`, as long as they control a valid `credential` with signing authority. Add a check to enforce that the `existing_schema` belongs to the `credential` in `process_change_schema_version`.

Remediation

Refactor the code to include the above checks.

Patch

Resolved in 44a58ee.

© 2025 Otter Audits LLC. All Rights Reserved.

Improper Validation Logic OS-SAS-SUG-01

Description

1. `process_create_attestation` currently lacks a check to ensure that the expiry timestamp is valid. Accepting an expired timestamp is semantically incorrect and should be restricted. Ensure `expiry == 0` (no expiration) or `expiry >= current_timestamp`. This prevents invalid attestations.
2. `CreateCredentialArgs::try_from_bytes` only checks if the total instruction data is at least 41 bytes, but it does not validate the actual content. An attacker may exploit this by setting the internal `name_len` or `signers_len` fields to zero and padding the rest with arbitrary data. To prevent this, the function should explicitly check that both the `name` and `signer` lengths are non-zero.

>_ *src/processor/create_credential.rs* RUST

```rust
#[inline]
pub fn try_from_bytes(bytes: &[u8]) -> Result<CreateCredentialArgs, ProgramError> {
    // The minimum expected size of the instruction data.
    // - name (5 bytes. 4 len, 1 char)
    // - signers (36 bytes. 4 len, 32 pubkey)
    if bytes.len() < 41 {
        return Err(ProgramError::InvalidInstructionData);
    }

    Ok(CreateCredentialArgs {
        raw: bytes.as_ptr(),
        _data: PhantomData,
    })
}
```

Remediation

Incorporate the above validations into the codebase.

Patch

Resolved in 44a58ee.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity OS-SAS-SUG-02

Description

1. `pda_utils::create_pda_account` incorrectly calculates required lamports utilizing `min_rent_space` even when it is smaller than the actual space needed. This may result in the account creation failing due to insufficient rent-exempt funding. The intent is for `min_rent_space` to act as a floor, not a fixed override. Modify the rent calculation, such that it is calculated utilizing the maximum of space and `min_rent_space`.

   >_ *src/processor/shared/pda_utils.rs* RUST

   ```rust
   /// Create a PDA account for the given seeds.
   pub fn create_pda_account<const N: usize>(
       payer: &AccountInfo,
       rent: &Rent,
       space: usize,
       owner: &Pubkey,
       new_pda_account: &AccountInfo,
       new_pda_signer_seeds: [Seed; N],
       min_rent_space: Option<usize>,
   ) -> ProgramResult {
       let signers = [Signer::from(&new_pda_signer_seeds)];
       let required_lamports = rent.minimum_balance(min_rent_space.unwrap_or(space)).max(1);
       [...]
   }
   ```

2. The `event_authority_pda` and `sas_pda` are deterministically derived from constant seeds and the program ID, implying they never change once deployed. Recomputing them in every instruction wastes compute units. Hard-code these values to for improved efficiency and readability.
3. The `CloseAttestationEvent` does not follow the anchor event CPI pattern and have the correct Event CPI discriminator.
4. The current implementation includes several cases where a user-supplied value is utilized to compute and dereference a raw pointer. This introduces unsafe memory operations, particularly since `from_raw_parts` also requires the region to be valid. If the pointer is null, unaligned, or the resulting slice exceeds allocated memory, it may result in undefined behavior. Refrain from directly utilizing unchecked user input for memory operations. The following list highlights the functions that may trigger such undefined behavior:
   - `create_credential`
   - `create_schema`
   - `change_authorized_signers`

© 2025 Otter Audits LLC. All Rights Reserved.

- `change_schema_version`
- `create_attestation`
- `create_tokenized_attestation`

Remediation

Implement the above-mentioned suggestions.

Patch

Resolved in 44a58ee.

© 2025 Otter Audits LLC. All Rights Reserved.

Unutilized/Redundant Code

OS-SAS-SUG-03

Description

1. `verify_pda` in `schema` (shown below) and `credential` checks whether a `credential` / `schema` account's address matches the expected PDA. However, it is redundant, as the account's ownership is already verified to belong to the program, and successful deserialization confirms it is a valid `credential` / `schema`. These two checks together effectively guarantee the account's authenticity. Moreover, since PDA derivation results in unnecessary computational overhead, it will be optimal to remove `verify_pda`.

>_ *program/src/state/schema.rs* RUST

```rust
pub fn verify_pda([...]) -> Result<(), ProgramError> {
    let (expected_schema_pda, _bump) = SolanaPubkey::find_program_address(
        &[
            SCHEMA_SEED,
            self.credential.as_ref(),
            self.name.as_ref(), // Convert Vec<u8> to UTF8 Array
            &[self.version],
        ],
        &SolanaPubkey::from(*program_id),
    );
    if acc_info.key().ne(&expected_schema_pda.to_bytes()) {
        log!("PDA Mismatch for {}", acc_info_as_str!(acc_info));
        return Err(ProgramError::InvalidAccountData);
    }
    Ok(())
}
```

2. In `create_attestation::data` (shown below), when creating a slice of `len`, there is no requirement to cast `len` as `usize` as it is already of the correct type. Similarly, in the `schema` structure, when serializing the `version` field, casting `version` as `u8` is unnecessary and should be removed.

>_ *src/processor/create_attestation.rs* RUST

```rust
pub fn data(&self) -> Result<&[u8], ProgramError> {
    // SAFETY: the `bytes` length was validated in `try_from_bytes`.
    unsafe {
        let len = self._data_len();
        let data_bytes = core::slice::from_raw_parts(self.raw.add(36), len as usize);
        Ok(data_bytes)
    }
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

Remediation

Remove the redundant and unutilized code instances.

Patch

Resolved in 44a58ee.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.